

Project 1 - Constituent Counter

Connor Fuglestad - LING473 - Summer Quarter 2026

Project Description

This program counts the syntactic constituent types in an annotated corpus. It takes a directory of any number of these files as input and reports how often each constituent type appears across all the files in the given directory.

Results

Constituent Type	Count
Sentences	2747
Noun Phrases	13141
Verb Phrases	7920
Ditransitive Verb Phrases	48
Intransitive Verb Phrases	5474

Approach

After a lot of research on the internet, I approached this in three steps:

1. Tokenize: I use a regular expression to split the raw file text into a list of meaningful pieces including open parens, close parens, and words.
2. Parse: I recursively use a function that recreates the tree structure from the list of tokens generated in the tokenize step. It reads a label after each open paren and then takes the children of that label (the next non "space" strings) until it finds a close paren. Since the children may themselves be trees, I chose a recursion strategy to handle this elegantly (thanks Stack Overflow!). It outputs nested Python tuples.
3. Count: I use another recursive function that goes through the parsed tree. It checks the label and increments the appropriate counter by string matching. For cases where it finds "VP", it additionally checks the immediate children to classify as intransitive or ditransitive VPs.

I chose recursion for the parsing and counting functions because trees are inherently recursive data structures. I could have alternatively used regex to search for specific strings like "(NP ",

and this is in fact how I first began, but I ran into problems when trying to search for the Ditransitive and Intransitive VPs. Recursion handles this much better.

Notes

Nested constituents are counted at every level they appear. That is, an NP inside another NP **BOTH** add 1 to the total. This allows me to use the recursion technique, since otherwise I would need to account for edge cases rather specifically and recursion would be much more difficult to implement. I believe this is in line with the instructions we were given.

Special Features and Design Decisions

- I only use standard Python libraries. No external packages were installed. I came across the python package *nltk* in my research for this project and apparently it can parse Penn Treebank notation in one line but I did not want to rely on installing an external library and, furthermore, I did not want to mask the logic of what I was doing behind an external library that handles it for me.
- I use exact string matching on labels. That means tags like "NP-SBJ" are ignored and automatically excluded. If we wanted these to be included, I could potentially use regex to search for "NP" within a string instead of using the exact match logic I went with. The directions said to ignore them, so I did.
- This program will process any file in the directory that is given to it. This could potentially cause long runtimes if there are a lot of files in the directory.

Limitations

- I think rather obviously, this is written specifically for Penn Treebank annotated content. It cannot handle things not in that format and if something is passed in a different format, it may error.

Goals Completed

I was able, after finding the recursion strategy in my research, to count all five required constituent types as specified. The program takes one file path as its only argument and prints results to standard output in the specified format.